

批次标准化 Batch Normalization

李宏毅《机器学习》2025 第五节：让训练地形更好走



基于李宏毅老师课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅机器学习深度学习课程 (Bilibili)

时长：约 31 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=14>

目录

1	为什么 Batch Normalization 出现在这里	3
1.1	本章小结	3
2	尺度差异如何改变 Error Surface	3
2.1	本章小结	4
3	Feature Normalization	4
3.1	本章小结	6
4	深层网络中为什么还需要 Normalization	6
4.1	本章小结	7
5	Batch Normalization 的计算图	7
5.1	本章小结	8
6	为什么还要 Gamma 与 Beta	8
6.1	本章小结	9
7	Testing 阶段怎么办	9
7.1	本章小结	10
8	BN 的经验效果与解释争议	11
8.1	Internal Covariate Shift 解释	11
8.2	更可靠的解释: 改变 Error Surface	12
8.3	本章小结	13
9	总结与延伸	13
9.1	延伸阅读	14

1 为什么 Batch Normalization 出现在这里

这节课接在“类神经网络训练不起来怎么办”的系列后面。前面几讲讨论过 learning rate、momentum、adaptive learning rate、loss function 等训练技巧，它们大多是在 optimizer 或 objective 上动手。Batch Normalization 则更像是直接改造网络内部的计算流程，让训练时面对的 error surface 更容易优化。

老师用“把山铲平”来描述这个想法：如果 loss landscape 在某些方向很陡、某些方向很平，gradient descent 就容易在峡谷里左右震荡，或者只能用很小的 learning rate 慢慢走。BN 的目标不是换一个 optimizer，而是让中间表示的尺度更稳定，使训练地形相对好走。

Batch Normalization 的核心问题

深度网络每一层都会改变资料分布。即使原始输入已经做过 normalization，经过前几层线性变换和 activation 后，后面层收到的 hidden features 仍可能有不同尺度、不同均值、不同方差。BN 尝试在训练过程中持续把这些中间特征拉回比较统一的尺度。

1.1 本章小结

BN 属于“改变网络内部地形”的训练技巧。它不是单纯预处理输入，而是在深层网络的中间层反复做 normalization。

2 尺度差异如何改变 Error Surface

先看最简单的线性模型：

$$y = b + w_1x_1 + w_2x_2.$$

如果 x_1 的值通常很小，例如 1、2；而 x_2 的值通常很大，例如 100、200，那么同样大小的参数改变会造成非常不同的输出变化。改变 w_1 一点点，对 y 的影响可能很小；改变 w_2 一点点，对 y 的影响可能很大。

从梯度角度看，

$$\frac{\partial L}{\partial w_i} = \sum_r \frac{\partial L}{\partial y^r} x_i^r.$$

当某个 dimension 的 x_i 尺度特别大，它对应方向的 gradient 和 loss curvature 都可能特别大。于是 error surface 变成长而窄的椭圆：一个方向平滑，另一个方向陡峭。

Changing Landscape

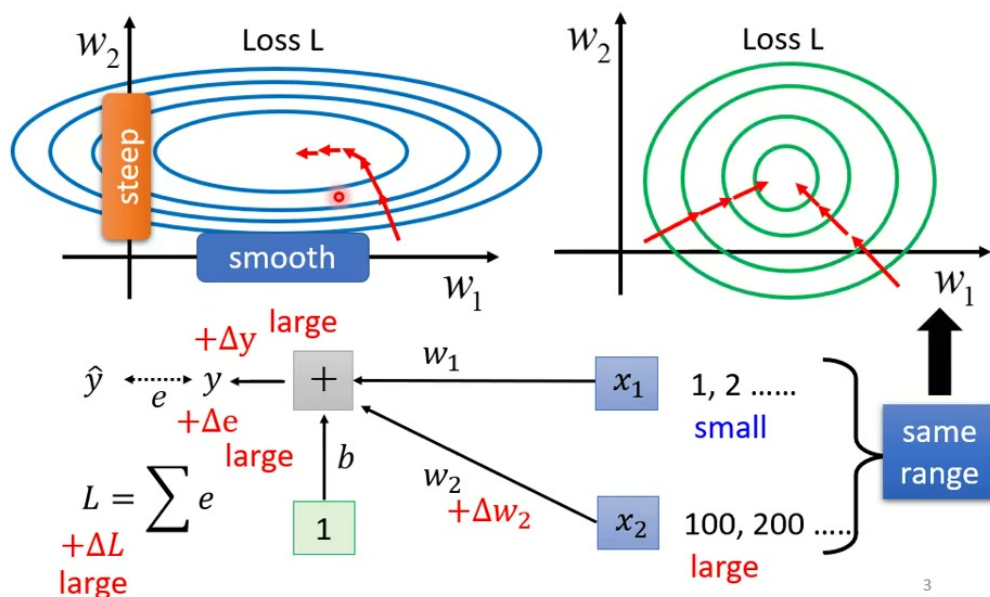


图 1: 不同输入维度尺度差异过大时, loss landscape 可能在不同方向呈现不同斜率; 把 feature 调到相近范围有助于让地形更圆滑。²

为什么椭圆地形难走

在狭长谷底中, 若 learning rate 太大, 参数会在陡峭方向来回震荡; 若 learning rate 太小, 平缓方向又前进太慢。Feature normalization 不保证 loss 变成完美圆形, 但它能减少某些纯粹由输入尺度造成的坏条件数。

2.1 本章小结

不同 dimension 的尺度差异会被参数更新放大成优化难度。把 feature 调整到相近均值和方差, 是改善训练地形的第一步。

3 Feature Normalization

给定训练资料

$$x^1, x^2, \dots, x^R,$$

²视频画面时间区间: 约 05:20-06:05; 字幕对应 “input feature 每一个 dimension 的 scale 差距很大” “产生不同方向斜率非常不同的 error surface”。

其中每个 x^r 都是一个多维向量。对第 i 个 dimension，可以计算训练集中该维度的平均值 m_i 和标准差 σ_i ：

$$m_i = \frac{1}{R} \sum_{r=1}^R x_i^r, \quad \sigma_i = \sqrt{\frac{1}{R} \sum_{r=1}^R (x_i^r - m_i)^2}.$$

然后把每个样本的该维度改写成

$$\tilde{x}_i^r = \frac{x_i^r - m_i}{\sigma_i}.$$

这样每个 dimension 的平均值约为 0，方差约为 1。

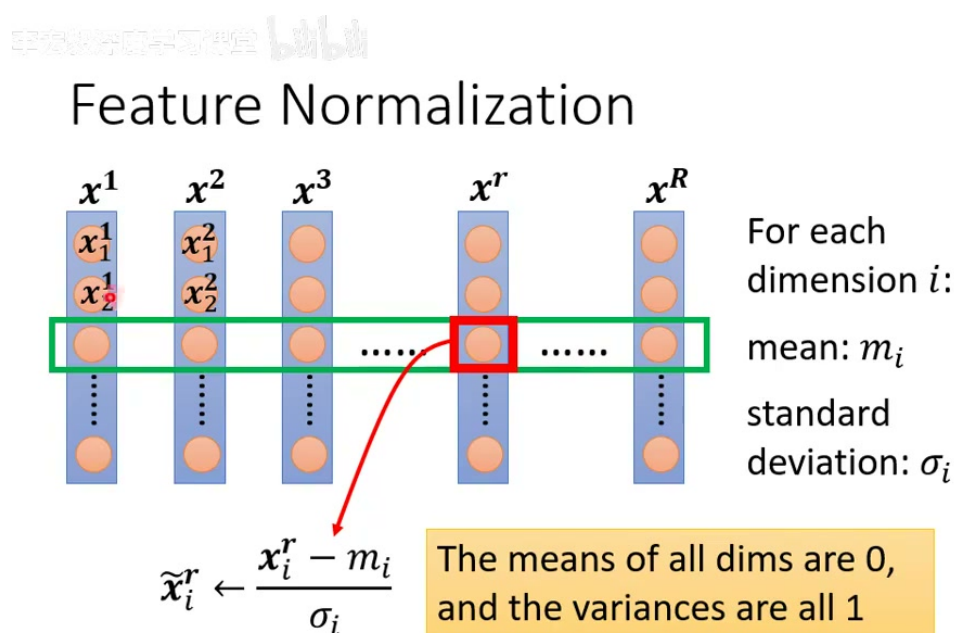


图 2: Feature normalization: 对每个维度分别减去平均值、除以标准差，使各维度平均约为 0、方差约为 1。⁴

许多深度学习作业或实作都会先对输入 feature 做 normalization。它有两个实际效果：

- 让各个 input dimension 对参数更新的影响不至于仅由尺度决定；
- 让模型更容易使用统一的 learning rate，而不是某些参数方向需要极小步长。

Normalization 不是资料泄漏的借口

训练时的平均值和标准差应该由训练集估计，再用于 validation 或 test。若用完整资料集、尤其包含测试资料来估计 normalization statistics，就会把测试集分布信息泄漏进训练流程。

⁴视频画面时间区间：约 07:50–08:25；字幕对应“减掉这个 dimension 算出来的 mean，再除掉 standard deviation”“平均是零”。

3.1 本章小结

Feature normalization 是 BN 的前置直觉：每个维度先对齐尺度，优化会更容易。但深度网络的问题不止在输入层。

4 深层网络中为什么还需要 Normalization

即使原始输入 $\tilde{x}$ 已经 normalization，经过第一层线性变换

$$z = W^1 \tilde{x}$$

之后， z 的各个维度仍可能有不同范围。再经过 activation 得到 a ，传给下一层 W^2 时，下一层看到的其实是新的 input。换句话说，每一层的 hidden representation 都可能重新产生尺度问题。

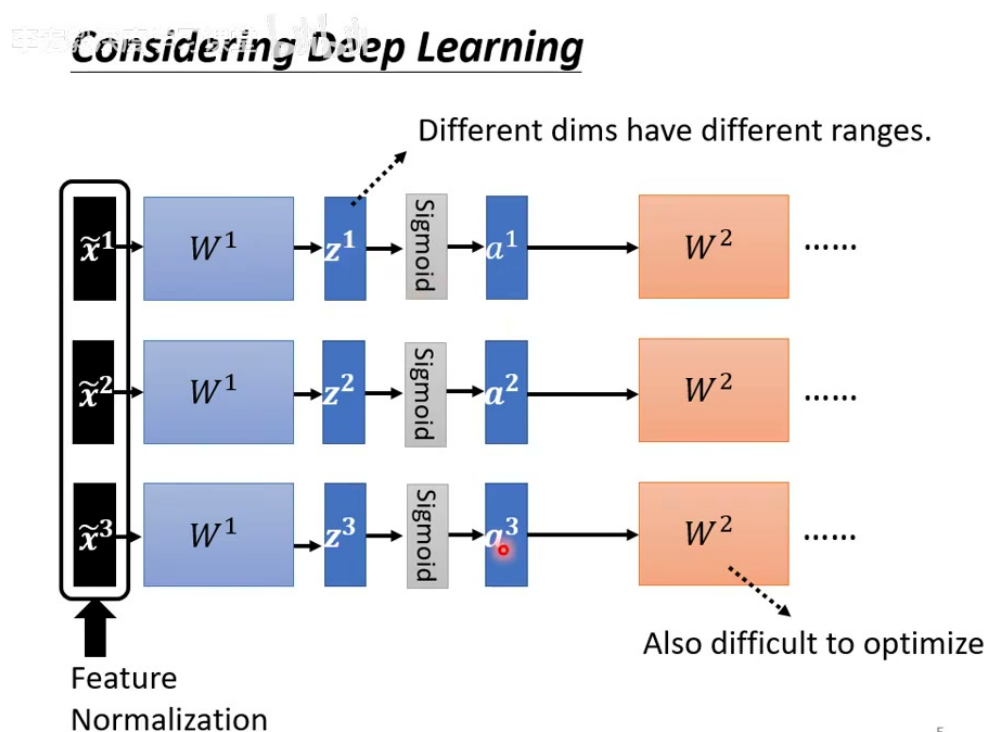


图 3: 原始输入做过 normalization 不代表 hidden features 也保持同样尺度；经过 W^1 与 activation 后，不同维度仍可能有不同范围。⁶

于是可以把 normalization 放在 hidden layer 中。常见选择是对 activation function 前的 pre-activation z 做 normalization：

$$\tilde{z} = \frac{z - \mu}{\sigma}.$$

之后再送进 activation function。若 activation 是 sigmoid，把 z 拉回 0 附近还有一个额外直觉：sigmoid 在 0 附近斜率较大，比较不容易进入饱和区。

⁶视频画面时间区间：约 10:45–11:25；字幕对应“对 Z 做 feature normalization 或对 A 做 feature normalization 都可以”“sigmoid 在零附近斜率比较大”。

BN 放在 activation 前还是后

课堂中说，实作上二者差异未必总是巨大；若使用 sigmoid，通常更推荐放在 activation 前，因为可以让输入 activation 的值靠近斜率较大的区域。现代 CNN 中常见顺序是 convolution、batch normalization、activation。

4.1 本章小结

深层网络每一层都会制造新的 feature distribution，因此只 normalize 原始输入不够。BN 把 normalization 嵌入网络内部，使 hidden features 也保持较稳定的尺度。

5 Batch Normalization 的计算图

BN 的关键是：hidden feature 的平均值和标准差不是对单一样本计算，而是对一个 mini-batch 计算。设某一层对一个 batch 产生 pre-activation

$$z^1, z^2, \dots, z^m,$$

其中 m 是 batch size。BN 先计算 batch mean 和 batch variance：

$$\mu_B = \frac{1}{m} \sum_{r=1}^m z^r,$$

$$\sigma_B^2 = \frac{1}{m} \sum_{r=1}^m (z^r - \mu_B)^2.$$

然后对 batch 中每个样本做 normalization：

$$\tilde{z}^r = \frac{z^r - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

这里 ϵ 是很小的数，用来避免除以零。

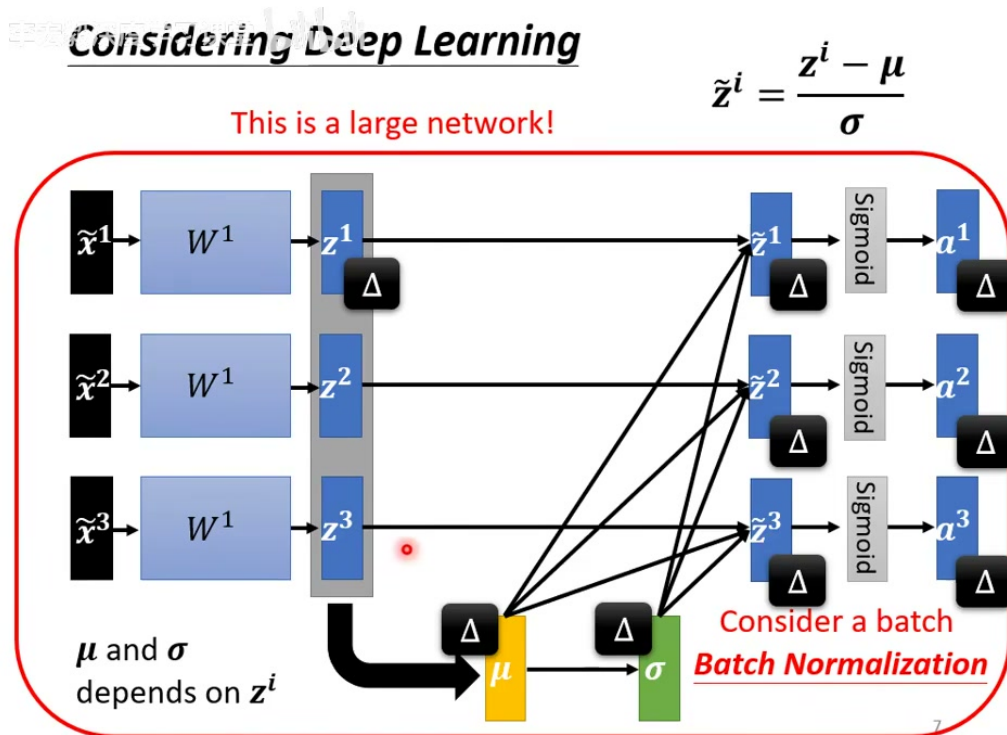


图 4: 在 deep network 中, μ 和 σ 由一整个 batch 的 hidden features 决定; BN 因此成为网络计算图的一部分。⁸

这个图很重要, 因为它说明 BN 不是对每笔资料独立做一个固定变换。某个样本的 normalized value 会受同 batch 中其他样本影响; 反向传播时, 梯度也会通过 μ_B 和 σ_B 连接 batch 内样本。

Batch size 太小会影响 BN

如果 batch size 太小, batch statistics 会很不稳定; 极端情况下 batch size 为 1, 方差几乎无法提供有意义的尺度信息。小 batch 场景常需要改用 Layer Normalization、Group Normalization, 或使用同步 BN 等技巧。

5.1 本章小结

BN 的 “batch” 不是名字装饰, 而是机制核心: 统计量来自 mini-batch, 因此训练时样本之间在 normalization 层发生耦合。

6 为什么还要 Gamma 与 Beta

如果只做

$$\tilde{z}^r = \frac{z^r - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

每个 dimension 都会被强行变成接近零均值、单位方差。这可能过度限制模型。也许某一层就是需要非零均值, 或者需要更大的尺度, 才能表示最适合任务的函数。

⁸视频画面时间区间: 约 17:20–17:55; 字幕对应 “你一定要有一个够大的 batch 才算得出 mu 跟 sigma”。

因此 BN 通常再加入两个可学习参数：scale γ 与 shift β ：

$$\hat{z}^r = \gamma \odot \tilde{z}^r + \beta.$$

其中 $\odot$ 表示 element-wise multiplication。 γ 和 β 的维度与 feature dimension 相同，训练过程中由 gradient descent 学出。

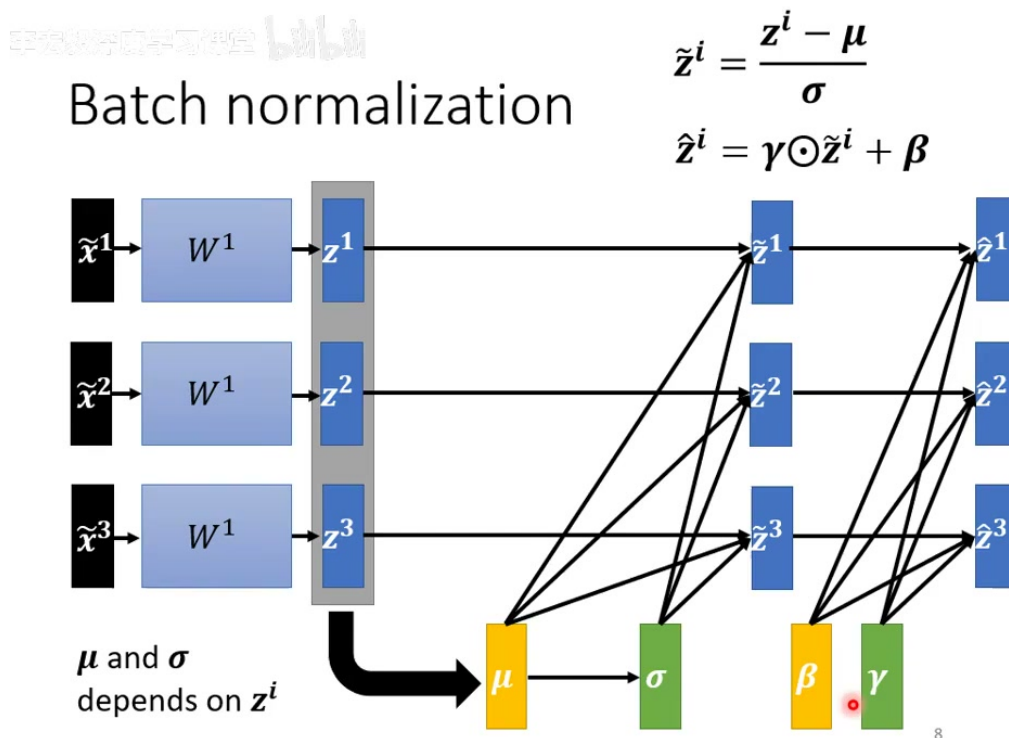


图 5: BN 先标准化，再用可学习的 γ 与 β 恢复模型需要的尺度和偏移。⁹

Gamma 与 beta 的意义

BN 不是强迫所有 hidden features 永远均值为 0、方差为 1；它只是先把尺度问题消掉，再把“需要什么尺度和偏移”交还给模型学习。若任务确实需要原来的分布，理论上 γ 与 β 可以把它学回去。

6.1 本章小结

BN 的完整训练时变换是 normalize 后再 affine transform。 γ 和 β 保留表达弹性，让 normalization 不至于成为过强约束。

7 Testing 阶段怎么办

训练时可以用 mini-batch 估计 μ_B 和 σ_B 。但 testing 或 inference 时，经常一次只来一笔资料。在线服务更不可能等 64 笔 request 凑成 batch 后再统一预测。因此 testing 阶段不能继续依赖当前

⁹视频画面时间区间：约 18:05-18:45；字幕对应“再乘上另外一个向量叫做 gamma”“再加上 beta”。

batch statistics。

常见解决方法是在 training 时维护 running statistics。每次处理一个 batch 得到 μ_t 与 σ_t 后，更新 moving average：

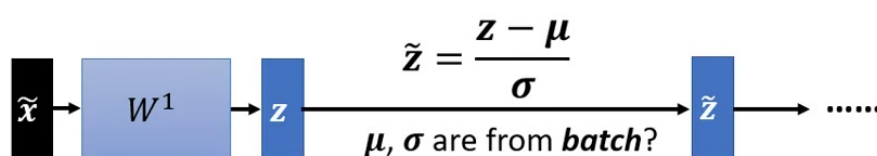
$$\bar{\mu} \leftarrow p\bar{\mu} + (1-p)\mu_t,$$

$$\bar{\sigma} \leftarrow p\bar{\sigma} + (1-p)\sigma_t.$$

Testing 时就使用训练过程中累计的 $\bar{\mu}$ 与 $\bar{\sigma}$ 。



Batch normalization – Testing



We do not always have batch at testing stage.

Computing the moving average of μ and σ of the batches during training.

$$\mu^1 \quad \mu^2 \quad \mu^3 \quad \dots \quad \mu^t$$
$$\bar{\mu} \leftarrow p\bar{\mu} + (1-p)\mu^t$$

图 6: Testing 阶段不一定有 batch；实作中通常使用 training 阶段累积的 moving average statistics。

11

在 PyTorch 中，BatchNorm 层会自动维护类似 `running_mean` 与 `running_var` 的统计量。调用 `model.train()` 时，BN 使用当前 batch statistics 并更新 running statistics；调用 `model.eval()` 时，BN 使用 running statistics，不再用当前 batch 估计。

常见实作坑

验证或测试前忘记调用 `model.eval()`，会导致 BN 继续使用当前 batch statistics，使结果依赖 batch 组成；训练恢复时忘记切回 `model.train()`，则 BN 不再更新统计量，也会影响训练行为。

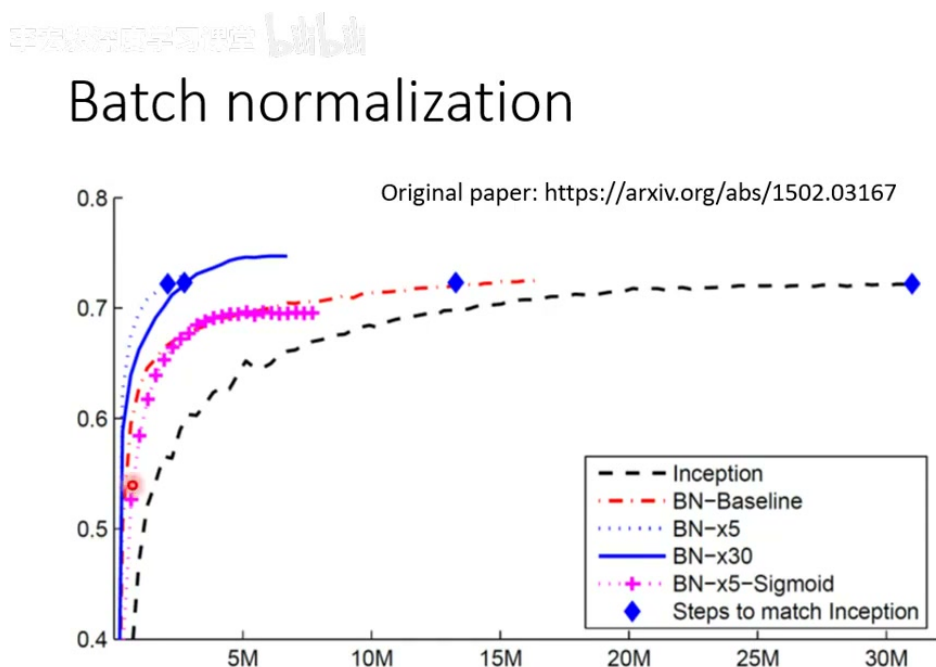
7.1 本章小结

BN 在 training 和 testing 阶段行为不同。Training 用当前 batch statistics 并更新 moving average；testing 用训练期间累积的 running statistics。

¹¹ 视频画面时间区间：约 21:50–22:30；字幕对应“training 的时候每一个 batch 计算出来的 mu 跟 sigma 都会拿出来算 moving average”。

8 BN 的经验效果与解释争议

原始 Batch Normalization 论文展示了非常直观的实验：加入 BN 后，模型可以更快训练；在某些设定中还可以使用更大的 learning rate。课堂中的图显示，红色和蓝色曲线比未使用 BN 的黑色虚线更快上升。



10

图 7: 原始 BN 论文中的实验曲线：加入 BN 后训练速度明显加快，并支持更大的 learning rate。¹³

8.1 Internal Covariate Shift 解释

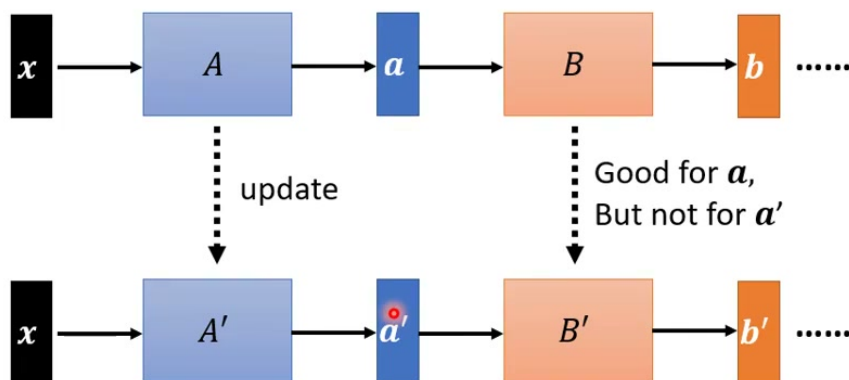
BN 最早常被解释为缓解 internal covariate shift。直觉是：前一层参数更新后，后一层看到的 activation distribution 会改变；后一层原本适合旧分布的参数，可能突然不适合新分布。BN 通过让 activation statistics 更稳定，使后一层面对的输入分布变化较小。

¹³ 视频画面时间区间：约 23:05–24:35；字幕对应“训练速度显然比黑色虚线快很多”“可以把 learning rate 设大一点”。

Internal Covariate Shift?

How Does Batch Normalization Help Optimization?

<https://arxiv.org/abs/1805.11604>



Batch normalization make a and a' have similar statistics.

Experimental results do not support the above idea.

11

图 8: Internal covariate shift 的原始解释: 前层更新会改变后层输入分布; 但后续实验并不完全支持“BN 主要靠减少分布漂移”这一说法。¹⁵

后续研究对这个解释提出质疑。实验发现, 即使没有明显减少 activation distribution 的变化, BN 仍然能改善训练。因此, “减少 internal covariate shift” 可能不是 BN 起作用的全部原因, 甚至不一定是主要原因。

8.2 更可靠的解释: 改变 Error Surface

课堂最后强调的观点是: 实验和理论分析更支持 BN 改变了 error surface, 让 loss landscape 更平滑、更容易优化。也就是说, BN 的效果可能不是简单让每层输入分布稳定, 而是改变了参数空间中 loss 的形状, 使 gradient descent 对 learning rate、初始化、局部曲率更不敏感。

¹⁵ 视频画面时间区间: 约 27:30–28:15; 字幕对应 “比较训练的时候 A 的分布变化” “不管有没有做 batch normalization, 它的变化都不大”。



Internal Covariate Shift?

How Does Batch Normalization Help Optimization?

<https://arxiv.org/abs/1805.11604>

Experimental results (and theoretically analysis) support batch normalization change the landscape of error surface.

and 12 of Appendix B.) This suggests that the positive impact of BatchNorm on training might be somewhat serendipitous. Therefore, it might be valuable to perform a principled exploration of the design space of normalization schemes as it can lead to better performance.

12

图 9: 后续论文认为 BN 的正面影响可能与 error surface 的改变有关, 并称其效果带有 serendipitous 的性质。¹⁷

Serendipitous 的含义

老师用盘尼西林的故事解释 serendipity: 它不是完全随机的好运, 而是在探索中意外发现了有价值的东西。BN 的提出也有类似味道: 最初解释未必完全正确, 但这个设计在经验上非常有效, 并推动了后续 normalization 方法的发展。

8.3 本章小结

BN 经验上显著改善训练速度和稳定性。早期常用 internal covariate shift 解释它, 后续研究更倾向于认为 BN 改变了 loss landscape, 使优化更容易。

9 总结与延伸

Batch Normalization 可以从三个层次理解。

第一, 作为 feature scale 修正。不同 dimension 的尺度差异会让 error surface 变得狭长崎岖; normalization 把均值和方差拉到相近范围, 让 gradient descent 更容易使用统一步长。

第二, 作为深层网络内部的动态 normalization。BN 不只处理输入, 而是处理 hidden features。训练时, 均值与方差来自 mini-batch, 因此 BN 层会让 batch 内样本发生统计耦合。

¹⁷ 视频画面时间区间: 约 28:35-29:20; 字幕对应 “可以改变 error surface, 让 error surface 比较不崎岖” “positive impact 可能是 serendipitous”。

第三，作为优化地形改造。BN 的好处不只是“减少分布漂移”，更可靠的理解是它让 loss landscape 更容易优化，通常支持更大的 learning rate，并提升训练稳定性。

实作记忆点

训练时 BN 用当前 batch 的 μ_B, σ_B ，并更新 running statistics；测试时 BN 用 running statistics。小 batch 时 BN 可能不稳定；需要时考虑 Layer Normalization、Instance Normalization 或 Group Normalization。

9.1 延伸阅读

后续阅读 normalization 方法时，可以比较 Batch Normalization、Layer Normalization、Instance Normalization、Group Normalization 的统计维度：它们的核心差异不是公式长得像不像，而是平均值和方差到底沿哪些 axis 计算。